

Teaching OpenGL 4.5/4.6 in Introductory Computer Graphics: A Hidden Sweet Spot in Graphics API Education Accessibility

Michael Shah

Yale University

New Haven, Connecticut, USA

michael.shah@yale.edu

Abstract

Vulkan, Direct3D 12, Metal, and WebGPU are among the most recently released graphics and compute APIs. Each of these forementioned graphics APIs allow programmers to more explicitly control how to submit work to the graphics device. However, the ability to have explicit programming control has created a steeper entry point for newcomers to graphics programming. The design of modern graphics APIs (designed for domain experts) has additionally created a tension for educators on what graphics API to teach in the classroom. The prior generation of Graphics APIs such as OpenGL 1.x through OpenGL 4.x, WebGL, and Direct3D 11 (and earlier) provided less verbose APIs to get new graphics programmers started, but these APIs may not give students the specific technology industry expects for entry level graphics programming positions. In this paper, I discuss my effort and philosophy on how to balance the tension of teaching beginners introductory computer graphics programming with the appropriate API. The solution I have found is by teaching OpenGL 4.5 as the minimum graphics API and introducing *Vulkan-like* programming abstractions. From our course experience, OpenGL 4.5 and OpenGL 4.6¹ provide a good balance (i.e. *sweet spot*) of allowing beginners to learn graphics programming from scratch in an API without any SDKs or proprietary frameworks, while still learning modern graphics programming techniques relevant for entry level graphics engineering jobs. After using OpenGL 4.5+, students are more prepared to take on Vulkan² and build their own higher level abstractions.

ACM Reference Format:

Michael Shah. 2026. Teaching OpenGL 4.5/4.6 in Introductory Computer Graphics: A Hidden Sweet Spot in Graphics API Education Accessibility. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Educator's Forum (SIGGRAPH Educator's Forum '26)*, July 19–23, 2026, Los Angeles, CA, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3799829.3812515>

1 Introduction

Computer graphics often is a popular course because of the potential to write *cool* games and simulations. Structuring a graphics

¹Note: We will refer to OpenGL 4.5 and 4.6 as OpenGL 4.5+ throughout this paper.

²Note: In this paper we compare OpenGL 4.5+ to Vulkan, but it is synonymous to the comparison of OpenGL 4.5+ to other modern graphics APIs such as Metal, Direct3D 12, and WebGPU.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGGRAPH Educator's Forum '26*, Los Angeles, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2552-4/26/07

<https://doi.org/10.1145/3799829.3812515>

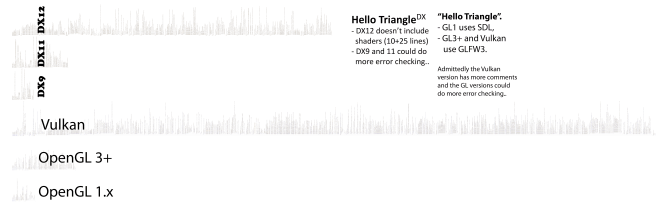


Figure 1: Provided is a visualization of the number of lines of code to setup a first triangle example. [Komppa 2023]

course to be accessible to a wide range of students (e.g. from pure CS students to tech+art students) and provide a reasonable on-ramp to a true beginner in graphics programming however can be difficult. Given the latest graphics APIs such as Vulkan, Metal, and Direct3D 12, beginners first must have at least intermediate familiarity with a programming language like C, C++, D, etc. Second, beginners will need to have or obtain a subset of linear algebra knowledge. Third, beginners new to graphics programming will be working in a graphics API that requires additional 'setup' for programming a graphics device.

The verbosity and number of steps of setting up a graphics programming API to a beginner is often perceived (whether correct or not) as making it much more difficult to program graphics. The explicit nature of the most recent set of modern graphics APIs allows but also demands the programmer to specify each individual 'knob' of their graphics devices for compute and graphics. This explicit control can be overwhelming to at least some subset of beginning graphics students. Debugging errors during this time can also be particularly challenging as they may be CPU, GPU, or system level bugs. Not to mention, that during this period beginners are also picking up new terminology and a new mental model for programming a GPU for the first time.

As an example Figure 1 provides a metric in terms of lines of code for an entry point exercise in drawing a triangle in several different graphics APIs. While lines of code is not a perfect metric, this provides a frame of reference on the perceived programming difficulty in each API to get started. An additional anecdote by an already experienced graphics programmer (with prior OpenGL graphics engine development experience) measuring their time to learn Vulkan from scratch as taking approximately three months echos this sentiment [Daler 2024]. Given that a single semester course is often spanning three to four months, the accessibility of graphics programming is becoming increasingly challenging with modern APIs.

Regardless of the perceived difficulty of API's like Vulkan for beginners, the Vulkan 1.0 Graphics API has been released for over

ten years and continues to get new updates [Khr 2016]. Industry adoption of Vulkan means that there will be continued need for new generations of graphics programmers to continue learning the state of the art in graphics programming [Ginsburg 2016]. Given industries need for graphics programmers, the question I pose to both academics and industry is:

Main Question: Is it now a requirement to teach Vulkan in an introductory graphics course?

In this paper, I address my thoughts, strategy, and anecdotes as to why OpenGL 4.5+ has currently been a successful approach to teaching students who will only have one semester of real-time graphics during their academic career, and that it is still appropriate to teach OpenGL.

2 The big ideas of modern real-time graphics programming

At colleges and universities the mission of a course (and degree program) is to provide a foundation of study such that students can build on that foundation for life, regardless of trends or specific tools that may go *out of fashion*. With that in mind, graphics can be taught in a variety of graphics APIs, pre-built frameworks, or graphics engines. Regardless of API, I have found three common central ideas of modern graphics programming as:

- (1) **Buffers:** Moving data (akin to a memcpy) between CPU and GPU memory.
- (2) **Shaders:** Write GPU programs using a domain specific language (e.g. glsl) and are compiled to execute on the GPU. This forms the *graphics and compute pipelines* that the GPU executes on during a draw or dispatch function call.
- (3) **Command Buffers:** Command buffers record all of the state (render flags, GPU buffers, graphics/compute programs) into a chunk of work that can be queued and executed.

Table 1 below illustrates how several versions of the OpenGL and Vulkan APIs support various modern features of graphics programming. Table 1 is meant to illustrate that OpenGL 4.5+ exercises to some degree all of the central three ideas of graphics programming, and provides support for many other more common programming tools in modern graphics that would also be available to Vulkan. In the next section I will highlight some of the important ideas, and pedagogically how they are useful in teaching and preparing students for a future in graphics.

2.1 OpenGL 4.5+ Benefits as an Entry Point to Graphics

Any version of OpenGL beyond 3.3 additionally stands as a reasonable starting point for modern graphics programming because it deprecates all of the fixed function pipeline functionality found in OpenGL 1.x. However, OpenGL 4.5+ (and in some cases earlier 4.x features) adds more features that have analogous behavior towards how Vulkan works, while maintaining the more friendly entry point to graphics programming.

- OpenGL 4.3 offers a Debug message callback to help students know if they have setup state correctly. Until students are familiar with GPU debuggers, this provides a simple way

Table 1: (1) Display Lists may be considered a similar concept. (2) - OpenGL 4+ offers indirect rendering which can be used to introduce the idea of packaging rendering state with draw calls. (3) glError and glGetError queries on the state machine. (4) A debug message callback is available in OpenGL 4.3. (5) Within glsl shader you may explicitly set uniform locations. (6) OpenGL 4.6 Offers SPIR-V precompiled shaders.

API/ Availability	OpenGL 1.x	OpenGL 3.3	OpenGL 4.5+	Vulkan
Buffers	(1)	x	x	x
Shaders		x	x	x
Command Buffers			(2)	x
Framebuffers		x	x	x
Debug Support		(3)	(4)	x
Explicit layout			(5)	x
Compute Shaders			x	x
SPIR-V			(6)	x
Concurrency				x

to quickly get feedback in the similar manner that Vulkan provides validation layers.

- OpenGL 4.3 and later gives students access to computer shaders. Compute is becoming increasingly important paradigm for students to learn (and pragmatically a marketable skill). Compute shaders can be used to mimic ray tracing shader pipelines available in Vulkan.
- OpenGL 4.5 provides Direct State Access (DSA) to allow simpler management of objects (i.e. `glCreateBuffers(n,buffer)`) by removing many bind steps. With DSA, we can think more closely to *grouping data* together in a local context. This helps to start removing the idea of the global OpenGL context *state-machine* paradigm which does not exist in Vulkan.
- Indirect rendering (OpenGL 4.0+) and multidraw (OpenGL 4.3+) operations introduce ideas of packing state with draw calls and again thinking about local contexts. Furthermore, efficiency gains of 'submitting groups' of work to the GPU to draw can help in aid in how GPU hardware consumes data.
- OpenGL does not have push constants, but we can create an abstraction using `glUniform*` commands to introduce this idea. This also helps lead into discussions of different types of storage and registers on GPU devices. OpenGL 4.3+ provides Shader Storage Buffer Objects (SSBO) to aid in thinking about where and how memory is stored in the GPU. SSBO's also introduce the concept of synchronization which is prevalent in Vulkan, but often handled in the OpenGL driver.
- OpenGL 4.6 allows for SPIR-V shader compilation which matches Vulkan's operation. Students can practice using explicit bindings of uniform locations and storing shader uniforms in blocks. If this feature is too difficult, students can scale back and use `#version 450` style glsl code.

To summarize, instructors should have some familiarity with Vulkan, and then they can setup abstractions or otherwise target

the latest features in OpenGL 4.5+ that match the model of programming in Vulkan. This provides a balance of preparing students for the future, but in a more friendly on-ramp into computer graphics (and potentially with lower hardware entry point). Kilgard makes a similar feature set suggestions in his presentation on migrating from OpenGL to Vulkan [Kilgard 2016]. From my current 1-semester course, the transition to Vulkan using a framework (e.g. [cha 2020]) or SDL_GPU [SDL 2025] at the very end also becomes feasible.

2.2 Potential Drawbacks of using OpenGL versus Vulkan

If students only have one course to learn graphics, there are some disadvantages of choosing OpenGL. Some of these disadvantages are also looking into the future where it may become clear which API *must* be used due to future hardware.

- The internet rumblings of OpenGL being *dead* can demoralize students learning a perceived *old* technology. Even with the recent mesh shader extension in 2025 to OpenGL it is perceived as an old API [Blumenkrantz 2025]. Furthermore, Apple's support of only OpenGL 4.1 may require additional creative setup of virtualization to get OpenGL 4.5+ features (e.g. either through virtualbox with mesa drivers, or VirtualGL on server machines).
- Unterguggenberger et al suggest that one can simply 'flatten' the learning curve and skip OpenGL by learning Vulkan through a framework in an introductory course [Unterguggenberger et al. 2023]. The advantage here is that students would leave with Vulkan experience and perhaps be seen as competitive. Though I may argue for use of any framework, one's understanding is at the mercy of understanding a specific frameworks model. Unterguggenberger et al also have a two semester course graphics sequence which may help in their curriculum design.
- Given enough time, the number of public learning resources available should continue to grow and assist in the learning process of Vulkan [62V 2022; Bailey 2023; Kenwright 2017; Overvoorde 2019]. This may eventually make Vulkan the *easier* API given enough effort or perhaps future Vulkan feature updates that provide better abstractions.
- Web searches for tutorial content on OpenGL 4.6 tutorials yield fewer results. Thus this may require upfront investment in instructors providing materials, or otherwise students being warned of the many versions of OpenGL code that they need to also filter.
- Teaching modern Graphics API in the future map more directly up front how GPU hardware works which can be a valuable lesson to students. This is a risk if OpenGL does not continue to get future extensions for a long enough time period.
- While we do not consider introducing concurrency in an introductory level graphics course, Vulkan provides a more friendly concurrency model that can be mapped to a discussion on CPU and GPU bound workloads which is slightly more awkward in OpenGL.

3 Conclusions and Future Work

In this paper I present why I believe OpenGL 4.5+ may be the *sweet spot* for making graphics education an accessible entry point while also preparing students for the entry level industry positions with current and future graphics APIs. After 8 years and 14 iterations of my introduction to real-time computer graphics course with OpenGL 3.3/4.1, my experience migrating to OpenGL 4.5+ has not only been easier to use (more consistent and debuggable) for students but OpenGL 4.5+ provides a better path forward to learning Vulkan.

Teaching OpenGL remains consistent with informal feedback from industry veterans who state that OpenGL 3.3+ is what they expect of new grads. I speculate in this rapidly changing graphics industry however, that all entry level engineers should still continue to learn a modern API like Vulkan. From my semester course, I have found that students can learn Vulkan after the semester by porting their OpenGL 4.5+ semesters work into Vulkan using their foundational graphics. The knowledge obtained from using OpenGL 4.5+ with Vulkan-like abstractions is enough to reduce the onramp to Vulkan and still keep an introductory graphics course – introductory.

For departments that have only one course on real-time graphics programming, my recommendation is to use OpenGL 4.5+ at a minimum if the compute resources are available. Given two semesters, it may be feasible to teach one course in OpenGL and the following course in Vulkan. Otherwise, using Vulkan for two semesters with a carefully curated ramp up period can be done. As an industry, we must keep in mind that many of us benefited and entered the industry by having a fast path to move pixels on the screen, and we should provide that onramp to as many students as possible.

References

- 2016. Khronos Releases Vulkan 1.0 Specification. <https://www.khronos.org/news/press/khronos-releases-vulkan-1-0-specification> [Online; accessed 2026-02-04].
- 2020. charles-lunarg/vk-bootstrap: Vulkan Bootstrapping Library. <https://github.com/charles-lunarg/vk-bootstrap> [Online; accessed 2026-02-06].
- 2022. (62) Vulkan Lecture Series - YouTube. <https://www.youtube.com/playlist?list=PLm1qTlJ6KsE1Jx5HV4sd2jOe3V1KMHgn> [Online; accessed 2026-02-06].
- 2025. SDL3/CategoryGPU - SDL Wiki. <https://wiki.libsdl.org/SDL3/CategoryGPU> [Online; accessed 2026-02-06].
- Mike Bailey. 2023. The Vulkan Computer Graphics API. In *ACM SIGGRAPH 2023 Courses* (Los Angeles, California) (SIGGRAPH '23). Association for Computing Machinery, New York, NY, USA, Article 19, 158 pages. doi:10.1145/3587423.3595529
- Mike Blumenkrantz. 2025. Mesh Shaders In The Current Year – Mike Blumenkrantz – Super. Good. Code. <https://www.supergoodcode.com/mesh-shaders-in-the-current-year/> [Online; accessed 2026-02-06].
- Elias Daler. 2024. How I learned Vulkan and wrote a small game engine with it. <https://edw.is/learning-vulkan/> [Online; accessed 2026-02-05].
- Karlsson Baldur Sekulic Dean Ginsburg, Dan. 2016. GDC Vault - Practical Development for Vulkan (presented by Valve Software). <https://gdcvault.com/play/1023577/Practical-Development-for-Vulkan-presented> [Online; accessed 2026-02-06].
- Benjamin Kenwright. 2017. Getting started with computer graphics and the vulkan API. In *SIGGRAPH Asia 2017 Courses* (Bangkok, Thailand) (SA '17). Association for Computing Machinery, New York, NY, USA, Article 5, 86 pages. doi:10.1145/3134472.3136556
- Mark Kilgard. 2016. Migrating from OpenGL to Vulkan. https://my.eng.utah.edu/~cs5610/lectures/Migrating_from_OpenGL_to_Vulkan.pdf [Online; accessed 2026-02-06].
- Jari Komppa. 2023. hellotriangle_ogl_dx.png (6666×20723). https://solhsa.com/temp/hellotriangle_ogl_dx.png [Online; accessed 2026-02-06].
- Alexander Overvoorde. 2019. Introduction - Vulkan Tutorial. <https://vulkan-tutorial.com/> [Online; accessed 2026-02-06].
- Johannes Unterguggenberger, Bernhard Kerbl, and Michael Wimmer. 2023. Vulkan all the way: Transitioning to a modern low-level graphics API in academia. *Computers and Graphics* 111 (2023), 155–165. doi:10.1016/j.cag.2023.02.001